



Universidad Autónoma de San Luis Potosí
Facultad de ingeniería
Tratamiento de Imágenes
Parcial 4
Ramírez Galván Manuel
Actividad 3



05/05/2025

Línea de Producción

Objetivo:

Diseñar, programar y simular en CoppeliaSim una celda de trabajo con las siguientes características:

1. **Sistema de transporte**
Se debe simular una **banda transportadora** que traslade cubos desde un **punto A (origen)** hasta un **punto B (final de línea)**.
2. **Generación de piezas**
Las piezas (cubos) deben:
 - Generarse automáticamente a intervalos regulares.
 - Asignarse un **color aleatorio**: verde o azul.
3. **Sistema de detección por visión**
 - A la mitad de la banda, debe haber un **sensor de visión** que capture en tiempo real el color del cubo.
 - La detección debe realizarse desde un script externo en **Python**, utilizando la API remota de CoppeliaSim y procesamiento con **OpenCV**.
4. **Sistema de clasificación**
 - Al final de la banda, debe instalarse una **placa desviadora** conectada a un **motor rotatorio (joint)**.
 - Según el color detectado por el sensor:
 - Si el cubo es **azul**, la placa debe girar para desviar el cubo hacia la **derecha**.
 - Si el cubo es **verde**, la placa debe girar para redirigir el cubo hacia la **izquierda**.
5. **Destino de piezas**
 - Las piezas deben caer en **contenedores separados** según su color:
 - Contenedor derecho: cubos azules.
 - Contenedor izquierdo: cubos verdes.

Pasos:

En la escena se colocó una banda transportadora simple, la cual al inicio tiene un cubo donde inicia el proceso y al final de la banda se encuentra una revoluta con una barrera para poder separar los cubos, fuera y al final de la banda se encuentran dos bases, estas de color verde y azul que representan los recipientes donde tienen que llegar a su respectivo color los cubos y arriba de la banda se encuentra un sensor de visión para poder ver y hacer la separación de las piezas.

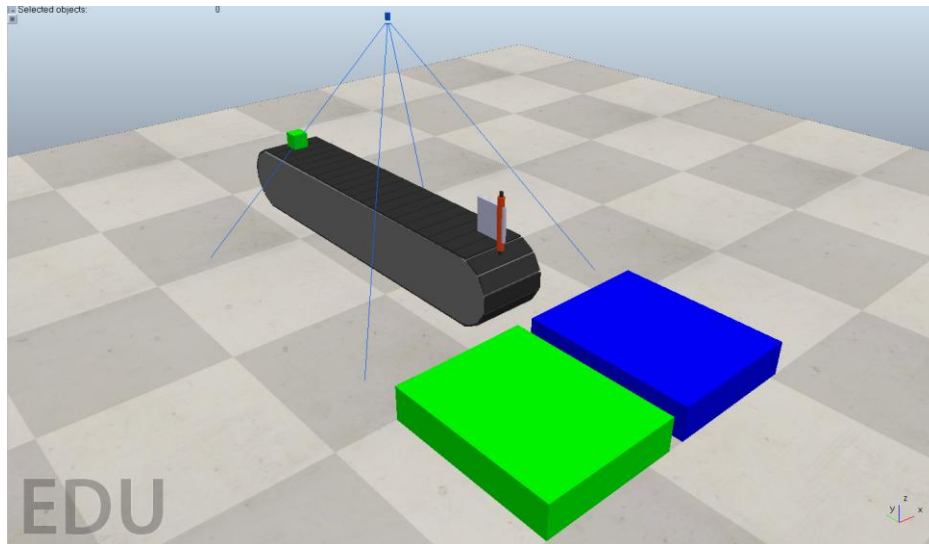


Imagen 1.- Imagen de la Escena

La herencia de la escena es simple para todos, los únicos que son hijos de otros son la barrera que va en la revoluta y el script que va en el dummy.

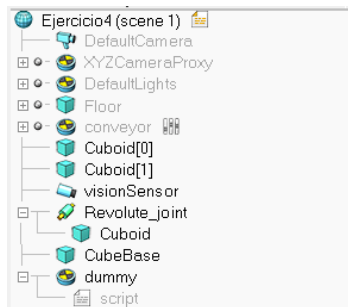


Imagen 2.- Imagen de la Herencia

Se utiliza un dummy para generar bloques de color verde y azul de forma aleatoria en la posición donde esta el cubo al inicio de la banda, esto cada cierto tiempo constante.

```

1 def sysCall_init():
2     sim = require('sim')
3     global cubeBase, counter
4     cubeBase = sim.getObject('/CubeBase')
5     counter = 0
6     pass
7
8 def sysCall_actuation():
9     global counter
10    counter += 1
11
12    if counter % 200 == 0:
13        import random
14        color = [0, 1, 0] if random.randint(0, 1) == 0 else [0, 0, 1]
15        position = [0.1, 1.125, 0.3]
16
17        newCube = sim.copyPasteObjects([cubeBase], 0)[0]
18        sim.setObjectPosition(newCube, -1, position)
19        sim.setShapeColor(newCube, None, sim.colorcomponent_ambient_diffuse, color)
20        sim.setObjectInt32Param(newCube, sim.shapeintparam_static, 0)
21        sim.setObjectInt32Param(newCube, sim.shapeintparam_respondable, 1)
22        sim.setObjectSpecialProperty(newCube, sim.objectspecialproperty_collidable)
23    pass

```

Imagen 3.- Código para Generar Cubos

En el programa se hace la conexión con Copelia, se carga la escena, se cargan los elementos que se van a utilizar los cuales son el sensor de visión y la revoluta, después se inicia la simulación y se crea una función la cual es para identificar objetos de color verde y azul, la cual dependiendo del que sea va a usarse después para mover la revoluta a una posición u otra para desviar el objeto a su respectiva base.

```
import time
import numpy as np
import cv2
from coppeliasim_zmqremoteapi_client import RemoteAPIClient

print('Iniciando programa de clasificación')
client = RemoteAPIClient()
sim = client.require('sim')

# Cargar escena
sim.loadScene('C:/Users/HUAWEI/Desktop/TRATAMIENTO/PARCIAL4/Ejercicio4.ttt')

# Obtener objetos
visionSensor = sim.getObject('/visionSensor')
joint = sim.getObject('/Revolute_joint')

# Activar motor y control loop desde el código
sim.setObjectInt32Param(joint, sim.jointintparam_motor_enabled, 1)
sim.setObjectInt32Param(joint, sim.jointintparam_ctrl_enabled, 1)

# Iniciar simulación paso a paso
sim.setStepping(True)
sim.startSimulation()

def detectar_color(img):
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

    lower_green = np.array([40, 70, 70], dtype=np.uint8)
    upper_green = np.array([80, 255, 255], dtype=np.uint8)

    lower_blue = np.array([100, 70, 70], dtype=np.uint8)
    upper_blue = np.array([130, 255, 255], dtype=np.uint8)

    mask_green = cv2.inRange(hsv, lower_green, upper_green)
    mask_blue = cv2.inRange(hsv, lower_blue, upper_blue)

    color_detected = 'ninguno'

    if cv2.countNonZero(mask_green) > 500:
        color_detected = 'verde'
    elif cv2.countNonZero(mask_blue) > 500:
        color_detected = 'azul'
    return color_detected, mask_green, mask_blue

while True:
    img, [resX, resY] = sim.getVisionSensorImg(visionSensor)
    img = np.frombuffer(img, dtype=np.uint8).reshape(resY, resX, 3)
    img = cv2.flip(cv2.cvtColor(img, cv2.COLOR_RGB2BGR), 0)

    color_detected, mask_green, mask_blue = detectar_color(img)

    if color_detected == 'verde':
        print('Color detectado: Verde')
        sim.setJointTargetPosition(joint, -35 * np.pi / 180)
    elif color_detected == 'azul':
        print('Color detectado: Azul')
        sim.setJointTargetPosition(joint, 35 * np.pi / 180)

    cv2.imshow('Sensor de visión', img)
    cv2.imshow('Mascara Verde', mask_green)
    cv2.imshow('Mascara Azul', mask_blue)

    key = cv2.waitKey(1) & 0xFF
    if key == 27:
        break

    sim.step()

sim.stopSimulation()
while sim.getSimulationState() != sim.simulation_stopped:
    time.sleep(0.1)

cv2.destroyAllWindows()
```

Imagen 4.- Código de Clasificación

Como resultado tenemos que se hace el reconocimiento de los objetos con su respectivo color.

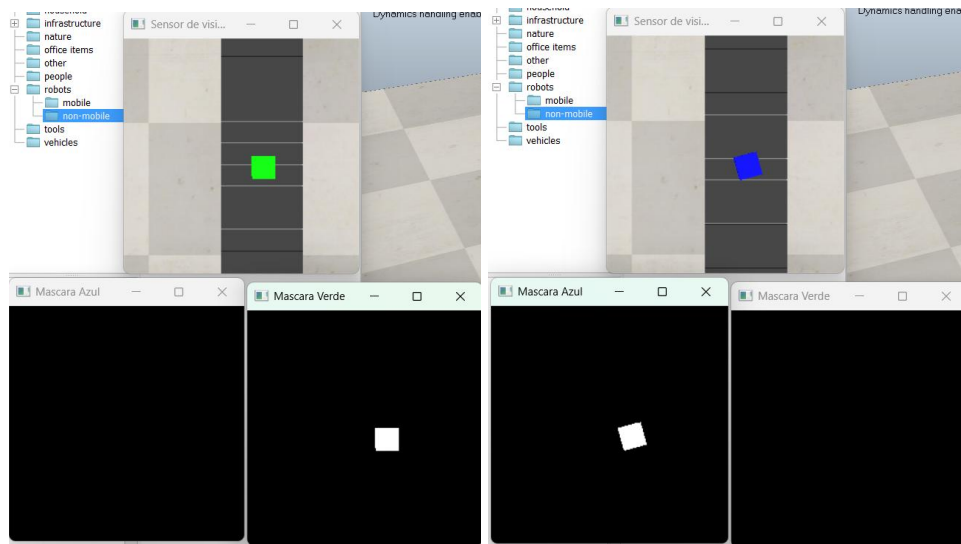


Imagen 5.- Detección de Objetos

Al detectar el color debe de moverse la revoluta para separar los cubos, lo cual no hace, se quedan atorados, hasta que llegan mas objetos y terminan moviendo a los de enfrente hasta tirarlos.

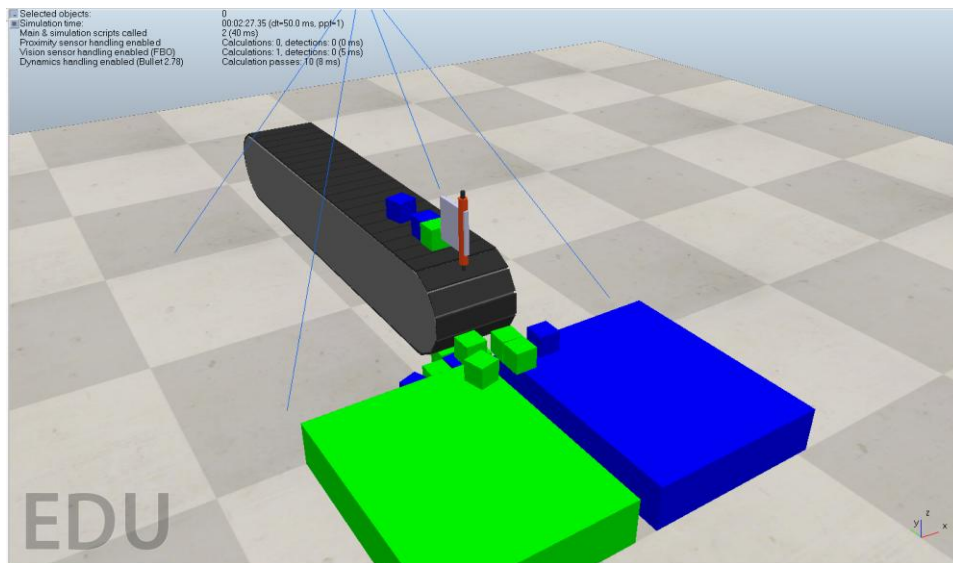


Imagen 5.- Clasificación de Objetos

Conclusiones

Se creo un proyecto en Coppeliasim en donde se hace la detección cubos de color verde y azul en una banda transportadora. Con ayuda de un sensor de visión y Python, se identifican los objetos, y un joint rotativo los clasifica según su color. Además, se agregó la generación de cubos para simular un flujo continuo. El único inconveniente fue que no se movía el joint a pesar de reconocer el cubo.